

Lecture Slides

Object Oriented Programming

Current Object, This Pointer, Copy Constructor

AGENDA

The agenda of today's lecture is:

1. Current Object in Member Function
2. This Pointer
3. Copy Constructors
4. Examples
5. Practice Problems

1. Current Object in Member Function

In C++, every non-static member function has access to the object that invoked it.

This object is referred to as the "current object." It allows member functions to operate on the data members of the object that called them.

This is particularly useful when working with multiple objects or when chaining function calls.

1.2 Member Function with Two Objects Example

```
class Distance {  
    int meters;  
public:  
    void set(int m) { meters = m; }  
    void add(Distance &d) { meters += d.meters; }  
    void show() {  
        cout << "Distance d1, d2;  
        d1.set(50);  
        d2.set(30);  
        d1.add(d2); // d1 is the current object  
        d1.show(); // Output: 80 meters  
    }  
};
```

2. The **this** Pointer

this pointer is an implicit parameter to all non-static member functions. It points to the object that invoked the function. It is especially useful when:

- Local variables shadow class members
- Return the current object from a function
- Chain multiple function calls

2.1. Resolving Variable Conflict

When a local variable has the same name as a class member, the `this` pointer helps distinguish between them. This avoids ambiguity and ensures the correct variable is accessed.

```
class Box {  
    int length;  
public:  
    void setLength(int length) {  
        this->length = length; // Resolves ambiguity  
    }  
};
```

2.2 Returning Current Object

Returning `*this` allows chaining of function calls. This is useful in fluent interfaces or builder patterns.

```
class Counter {  
    int count;  
public:  
    Counter(int count) { this -> count = count;}  
    Counter& increment(){ count++; return *this; }  
    void show() { cout << "Count: " << count << "\n";  
}  
};
```

`Counter c(5);
c.increment().increment().show();
// Output: 7`

3. Copy Constructors

A copy constructor is a special constructor used to create a new object as a copy of an existing object. It is invoked when:

- An object is passed by value
- An object is returned from a function
- An object is explicitly copied

Syntax:

```
ClassName(const ClassName &obj);
```

Used when:

- Passing object by value
- Returning object from function

3.1. Basic Copy Constructor Example

This example shows how to define a copy constructor that duplicates the values of another object.

```
class Point {  
    int x, y;  
public:  
    Point(int a, int b) : x(a), y(b) {}  
    Point(const Point &p){    x = p.x;        y = p.y;        }  
    void show(){cout <<"(" << x << ", " << y << ")\n";}  
};
```

3.2 Copy Constructor in Action

Here, the copy constructor is automatically called when p2 is initialized using p1.

```
Point p1(3, 4);
```

```
Point p2 = p1;    // Copy constructor called
```

```
Point p3(p1);    // Copy constructor called
```

```
p2.show();       // Output: (3,4)
```

```
p3.show();       // Output: (3,4)
```

4.1 Chainable Setters Using this Example

This example shows how to use this to return the current object and chain multiple setter calls.

```
class Person {  
    string name;  
    int age;  
public:  
    Person& setName(string n){ name=n; return *this;}  
    Person& setAge(int a) { age = a; return *this; }  
    void show() {  
        cout <<name<<" is "<< age << " years old\n"; }  
};
```

4.2 Copy Constructor with Object Array

This example demonstrates how a copy constructor works when copying objects in an array or collection.

```
class Book {  
    string title;  
public:  
    Book(string t) { title = t;}  
    Book(const Book &b) { title = b.title; }  
    void show() {  
        cout << "Title: " << title << "\n";  
    }  
};
```

```
Book b1("OOP Concepts");  
Book b2 = b1;  
b2.show(); // Output: OOP Concepts
```

5. Large Integrated Example

This example combines this pointer, copy constructor, and object interaction in a banking context.

```
class Account {  
    string name; int bal;  
public:
```

```
Account a1("Ali",1000), a2("Sara",500);  
a1.deposit(200).transfer(a2, 300);  
a1.show(); // Ali: 900  
a2.show(); // Sara: 800
```

```
Account(string n, int b){ name = n, bal = b; }  
Account(const Account &a) {name=a.name; bal=a.bal; }  
Account& deposit(int amt) {bal += amt; return *this;}  
void transfer(Account &to, int amt) {  
    if (bal >= amt) { bal -= amt; to.bal += amt; } }  
void show() { cout << name << ": " << bal << "\n"; }  
};
```